

OPERATIONS GUIDE · DOMAIN
RELEASE AND LAUNCH

KEJA APP

The keja.app domain claim, confirmed at the API level: which Netlify account holds it, the five-minute self-service release, the support fallback, and the one-command automated finish.

Operations guide (v2 — confirmed diagnosis)

A CHACADOM INVESTMENTS VENTURE

10 SEPTEMBER 2026

KEJA AI · KEJA.APP

Table of Contents

1 What the Error Means — and the Confirmed Diagnosis2

 The API forensics (what was checked and found)2

2 Path A — Release It Yourself (~5 minutes)3

3 Path B — Netlify Support Releases It4

4 Attaching keja.app After the Release4

5 Post-Launch Verification Checklist5

6 Tooling Reference6

7 Second Blocker — Stuck Operational-Credits Flag6

1 What the Error Means — and the Confirmed Diagnosis

When adding `keja.app` to the Netlify site, verification fails with: *'keja.app or one of its subdomains is already managed by Netlify DNS on another team.'* This guide now carries a confirmed, API-level diagnosis of exactly which account holds the claim, the five-minute self-service release path, and the one-command automated finish. It is written against the verified state of 10 September 2026.

The message says a **different Netlify account** already holds a claim on `keja.app`. Netlify enforces a one-owner rule: an apex domain can be claimed by only one Netlify account at a time, no matter who owns the registrar account. The claim is an internal Netlify record—most often left behind when the domain was once added to a site or DNS zone in another account. Crucially, the claim does not need active nameservers to keep blocking you: it lives in Netlify's database, not in the DNS system. Your DNS side is already perfect—nameservers are Spaceship's (`launch1` and `launch2.spaceship.net`), the apex A record points to `75.2.60.5` (Netlify's load balancer), and `www` is a CNAME to `keja-ai.netlify.app`.

The API forensics (what was checked and found)

A full sweep was run through the Netlify API with the deploy token of the account that owns `keja-ai` (login `torv54@gmail.com`, team `'Victor' / gadda00`, account id `68331ef7ea60d8e7aedec052`). Every team, every site, every domain-bearing field on each site object, every DNS zone and the account audit trail were enumerated. The account contains four sites—`keja-ai`, `chacadom`, `ecoawardsafrica` and `busara-ai`—and **none of them claims `keja.app`**: `chacadom` holds only `chacadom.com`, `busara-ai` holds only `busaraai.com`, and no `keja.app` DNS zone exists in this account. The attach attempt itself returned the decisive answer:

```
PATCH /api/v1/sites/{keja-ai} { "custom_domain": "keja.app" }
422 Unprocessable Entity
{ "custom_domain": [ "is owned by another account",
  "must be unique (keja.app, fb3b99bc-55cd-4552-a3a4-4b378448aa47)" ] }
```

Netlify itself names the owner: `keja.app` is claimed by account **`fb3b99bc-55cd-4552-a3a4-4b378448aa47`**—a different login from the one that owns `keja-ai`. This is almost certainly a **second account of yours**: GitHub OAuth, Google and email/password sign-ins each create separate Netlify accounts, even for the same email address. When the domain was first being linked, the 'Add domain' step ran while the browser was logged into that other identity, creating a zone there. The table below summarises the verified state.

Check	Result
Sites visible to the deploy token	4: <code>keja-ai</code> , <code>chacadom</code> , <code>ecoawardsafrica</code> , <code>busara-ai</code>

Check	Result
keja.app on any site (custom_domain, aliases, branch/preview)	None—chacadom only holds chacadom.com
Netlify DNS zones in this account	Only busaraai.com—no keja.app zone
DNS for keja.app (registrar side)	NS launch1/launch2.spaceship.net; A 75.2.60.5; www CNAME keja-ai.netlify.app
Claiming account (from the 422 error)	fb3b99bc-55cd-4552-a3a4-4b378448aa47—a second login

Table 1.1 — Confirmed state as of 10 September 2026.

2 Path A — Release It Yourself (~5 minutes)

Because the claiming account is almost certainly your own second login, this is the recommended path and needs nobody's permission but yours.

- **Step 1—Find the other login.** At app.netlify.com, log out, then try each identity you own: Continue with GitHub, Continue with Google, and any other email/password. In each, check the Teams you land in—you are looking for one that is **not** 'Victor' (the keja-ai team). Team settings → General shows the account id; the claiming one ends in 8448aa47.
- **Step 2—Delete the DNS zone there.** In the claiming account: team → Domains (the team-level DNS page). If keja.app is listed, open it → Options → **Delete DNS zone**. The zone is dormant—live nameservers are Spaceship's—so deleting it breaks nothing.
- **Step 3—Check that account's sites too.** If any site there lists keja.app or www.keja.app in Site configuration → Domain management, remove those entries (Options → Remove domain).
- **Step 4—Run the automated finish.** Repo → Actions → **Fix keja.app domain (Netlify)** → Run workflow → mode **fix**. It attaches keja.app (primary) + www.keja.app, provisions the Let's Encrypt certificate, enables Force HTTPS and polls until https://keja.app answers 200.

Fully automated alternative for Step 1-3: create a Netlify personal access token in the claiming account, add it as the NETLIFY_CLAIM_TOKEN secret in the repo, then run the same workflow with use_claim_token + release_only checked (it releases the claim via API), and finally re-run with mode=fix using the normal token. No browser needed.

3 Path B — Netlify Support Releases It

If the claiming account turns out not to be yours—cannot be found among your logins—Netlify support must release it. This is a routine request with a defined process.

- **Step 1—Open a support ticket.** From the app: Support → contact form. Subject: 'Domain claimed by another account—release request for keja.app'. Include the claiming account id fb3b99bc-55cd-4552-a3a4-4b378448aa47 so support can locate the stale claim instantly.
- **Step 2—Prove registrar ownership.** The WHOIS/RDAP registrant information for keja.app (registered 8 September 2026 at Spaceship), the purchase date and method, and a screenshot of the Spaceship dashboard showing the domain in your account.
- **Step 3—Demonstrate live DNS control.** Offer to add a temporary TXT record Netlify specifies at your Spaceship DNS panel on request. Being able to change DNS at will is conclusive ownership evidence and typically shortens the process to a few business days.
- **Step 4—Ask for the release explicitly.** Netlify will attempt to contact the claiming account, then release the claim if they do not respond or cannot justify it.

While you wait, nothing is blocked operationally: the site serves correctly on keja-ai.netlify.app, deploys run green on every push, and the DNS A record is already aimed at Netlify's load balancer. The moment the claim is released, Chapter 4 completes in under ten minutes. If support stalls beyond a week, escalate by replying on the same thread and cite the verified DNS state in Table 1.1.

4 Attaching keja.app After the Release

The automated route is one click: Actions → Fix keja.app domain (Netlify) → mode fix. It performs every step below, retries with backoff, and verifies the result. The manual equivalent:

- **Step 1—Add the apex domain.** Netlify → keja-ai site → Site configuration → Domain management → Add a domain → keja.app. Accept the prompt to make it the primary domain.
- **Step 2—Add www.** Add www.keja.app to the same site, then in Domain management set www to redirect to the apex (apex-primary is recommended for a brand like keja.app).
- **Step 3—Confirm DNS records at Spaceship.** Keep the apex A record 75.2.60.5, and set www as a CNAME to keja-ai.netlify.app. Green checks appear within minutes for the A record, up to an hour for the CNAME.
- **Step 4—HTTPS.** Let's Encrypt certificates are provisioned automatically once the domain verifies; then force HTTPS (Domain management → HTTPS → Force HTTPS).

```
# Verification commands (any terminal):  
dig NS keja.app +short           # launch1/launch2.spaceship.net
```

```
dig A keja.app +short          # 75.2.60.5
dig CNAME www.keja.app +short # keja-ai.netlify.app
curl -I https://keja.app      # HTTP/2 200 + netlify edge headers
```

One subtlety worth knowing: because you are staying on Spaceship nameservers rather than delegating to Netlify DNS, the Netlify DNS-check screen may show a gentle recommendation to 'use Netlify DNS'. That is optional. Keeping Spaceship DNS with A and CNAME records is fully supported, keeps registrar independence, and matches the already-verified configuration in Table 1.1. Change nothing unless you have a reason.

5 Post-Launch Verification Checklist

Run through this once keja.app answers. Every item is a one-command or one-click check, and together they prove the launch end-to-end.

#	Check	Expected result
1	curl -I https://keja.app	HTTP/2 200 with Netlify edge headers
2	curl -I https://www.keja.app	301 redirect to https://keja.app
3	Manifest at https://keja.app/manifest.webmanifest	Serves JSON; PWA installs from the canonical domain
4	Service worker at https://keja.app/sw.js	VERSION stamped (content hash), Cache-Control must-revalidate
5	Security headers on https://keja.app	X-Frame-Options DENY and X-Content-Type-Options nosniff present
6	Install on a real Android (Chrome)	Beforeinstallprompt fires; app installs full-screen with maskable icon
7	Install on a real iPhone (Safari)	Share → Add to Home Screen; branded launch screen on cold start
8	Offline behaviour	Airplane mode after first load: app shell and visited views still open

Table 5.1 — Eight checks that prove the canonical launch.

If any check fails, the likely culprits are narrow and checkable: DNS propagation (items 1-2—re-run dig after thirty minutes), missing www CNAME (item 2), or a stale service worker on a previously-visited device (items 4 and 8—a hard refresh and one re-open clears it, because the cache version is content-hashed and self-evicts on the next activate). Everything else already passed the same checks on the keja-ai.netlify.app domain during the September 2026 verification pass, so the canonical domain inherits a known-good bundle.

6 Tooling Reference

Everything described above is wired into the repository as repeatable tooling—no step depends on tribal knowledge or a one-off terminal session.

Artifact	Purpose
scripts/netlify-domain-fix.mjs	Zero-dependency Node script. <code>MODE=discover</code> inventories teams, sites, zones and claims (read-only). <code>MODE=fix</code> releases stale claims, attaches keja.app + www, provisions SSL and verifies. <code>MODE=probe</code> dumps raw endpoint diagnostics. <code>RELEASE_ONLY=1</code> releases without attaching, for use with the claiming account's token.
.github/workflows/fix-domain.yml	Manual workflow runner exposing the modes above, with optional <code>NETLIFY_CLAIM_TOKEN</code> secret for the cross-account release flow.
docs/DOMAIN_CONFLICT_FIX.md	This guide in markdown, kept current with the repo.

Table 6.1 — Domain tooling shipped in the repository.

7 Second Blocker — Stuck Operational-Credits Flag

While verifying the deploy pipeline on 10 September 2026, production deploys began failing with a second, unrelated Netlify-side fault.

```
POST /api/v1/sites/{keja-ai}/deploys
{"error": "Account credit usage exceeded - new deploys are blocked until credits are added"}
```

This is **not** an actual credit shortage. The Netlify API for this account reports `type=Free` with `credits included=300` and `used=0`, and no usage exceeded anywhere—yet production deploys are blocked. This matches a **known Netlify bug from July to September 2026** (the 'stuck operational-credits flag'), reported by many Free-plan teams on the Netlify forums—one report matches this case word-for-word, including the `used=0` reading. Three facts characterise the state: published sites stay live (keja-ai.netlify.app serves normally; only *new* production deploys are paused); draft deploys continue to work, the block is production-only; and only Netlify support can reset the flag—there is no self-serve path and it does not self-clear within the billing cycle.

The recommended action: include the flag reset in the same support ticket as the domain release from Chapter 3, or file it standalone—'Please reset the stuck operational-credits flag for team gadda00 (account 68331ef7ea60d8e7aedec052). The API reports `credits included=300`, `used=0`, yet production deploys fail

with Account credit usage exceeded. This matches the known Jul-Sep 2026 flag bug.' Until the reset, every push to main still builds and passes all quality gates on GitHub; only the final deploy step is paused, and it now exits with a self-diagnosing message pointing at this chapter. Once support resets the flag, re-run the latest failed Deploy workflow and the newest commit ships immediately.